

Orbit Wars — v12 PPO + GAE

The shipped on-policy learner: GPU-side GAE on the shaped reward, the clipped surrogate with PPO/DAPO/GSPO knobs, PopArt value targets, the BC-stability guardrails, and the optional UNREAL auxiliary reward-prediction head

Source of truth: `orbit_wars_v12/rollout.py`, `ppo.py`, `policy.py`

2026-06-19

Abstract

This is document 2 of four and describes the **default** learner: PPO with Generalized Advantage Estimation (`cfg.ALGO="ppo"`). It assumes document 1 (`v12_setup.tex`) for the game, the `GpuEnv` rollout, the observation encoding, the gated-allocation action, and especially the reward — PPO learns from exactly the reward defined there. Here we document only the PPO-specific machinery: how the advantage and value targets are built in `collect_ppo`, the clipped-surrogate update in `ppo_update` (including the DAPO and GSPO knobs), the value-warmup / LR-warmup / KL guardrails that make PPO-after-BC stable, and the optional auxiliary reward-prediction head. The last section addresses head-on whether that head “replaces GAE” (it does not, and we explain precisely why a reward head cannot).

Contents

1	What PPO adds to the shared loop	1
2	Advantage and value targets (<code>collect_ppo</code>)	2
3	The clipped-surrogate update (<code>ppo_update</code>)	2
3.1	BC-stability guardrails	3
4	The auxiliary reward-prediction head (<code>AUX_REWARD_PRED</code>)	3
4.1	Does the aux head “replace GAE”? No — and here is why a reward head cannot . .	4
4.2	Caveat	4
5	The win-bet head (<code>AUX_WIN_BET</code>) — a stronger aux target	4
6	Config-knob map	5

1 What PPO adds to the shared loop

The actor-critic network of document 1 has a **value head** (the PopArt critic) that PPO trains and uses as a learned baseline; GRPO (document 3) leaves it untrained. The per-iteration loop in `train.py` is shared: PFSP-sample league opponents, roll out B envs with `collect_ppo`, call

`ppo_update`, update Elo. PPO’s contribution is (i) the **GAE** advantage and value targets computed at the end of the rollout, and (ii) the **clipped surrogate + value loss** update. Key hyperparameters (`config.py`):

γ GAMMA	0.997	λ GAE_LAMBDA	0.98
clip CLIP	0.2	VF_COEF	0.5
ENT_COEF (anneal)	0.005 $\rightarrow$ 0.002/300it	MINIBATCHES	64
UPDATE_EPOCHS	3	MAX_GRAD_NORM	0.5
KL_TARGET	0.05	LOGRATIO_CLAMP	4.0
LR	10^{-4}	ADAM_EPS	10^{-5}

2 Advantage and value targets (`collect_ppo`)

During the rollout, each step stores the critic value $V(s_t) = \text{val_buf}[t]$ (PopArt-denormalized in `act`), the shaped per-step reward $r_t = \text{rew_buf}[t]$ (document 1, divided by `PPO_REWARD_SCALE`), a `done` flag (terminal on capture-out or death), and an `alive` mask. After the loop the terminal outcome payment $O/\text{PPO_REWARD_SCALE}$ is scattered onto each env’s last active step, so r_t carries both the dense shaping and the sparse game result.

Bootstrap. A final forward gives $V(s_T)$ for envs still active at the horizon: `bootstrap` = $V(s_T) \cdot \text{active}$ (zero for envs that already terminated — no spurious bootstrap past a decided game).

GAE recursion (on-GPU, no host sync). With $\delta_t = r_t + \gamma V(s_{t+1})(1 - \text{done}_t) - V(s_t)$, the truncated GAE is accumulated backwards:

$$A_t = \delta_t + \gamma \lambda (1 - \text{done}_t) A_{t+1}, \quad \hat{R}_t = A_t + V(s_t),$$

masked by `alive` at every step (finished envs contribute nothing). $\hat{R}_t$ is the value *target*; A_t is the policy advantage. The whole recursion runs as tensor ops on the device — no `.cpu()`, no `synchronize`.

Advantage normalization. On the PPO path the flattened advantage is standardized over the live transitions, $A \leftarrow (A - \bar{A})/(\sigma_A + 10^{-8})$, before it enters the surrogate. (The `unit_scaled` and `ADV_TARGET_RMS` branches in the same code block are GRPO-only and never fire for `ALGO="ppo"`.) Value targets $\hat{R}_t$ are stored raw in `tb["returns"]` and normalized by PopArt inside the update.

3 The clipped-surrogate update (`ppo_update`)

`ppo_update` runs `UPDATE_EPOCHS` passes over `MINIBATCHES` random minibatches, each re-evaluating the policy on the stored transitions and taking one Adam step. All logging stats are accumulated on-GPU and flushed with a *single* host sync at the end.

Importance ratio. For a minibatch, `evaluate` returns the current log-prob $\log \pi(a | s)$ (summed over owned sources, document 1). The log-ratio against the stored behaviour log-prob is

$$\rho = \exp(\text{clip}(\log \pi - \log \pi_{\text{old}}, \pm \text{LOGRATIO_CLAMP})).$$

[`GSPO / RATIO_LENGTHNORM`] optionally divides the log-ratio by the number of owned planets before exponentiating, so the ratio measures policy change *per decision* rather than a product over a variable planet count (whose variance otherwise trips the clip/KL unevenly). Default off.

Clipped surrogate.

$$L^{\text{pol}} = -\mathbb{E}[\min(\rho A, \text{clip}(\rho, 1 - \text{CLIP}, 1 + \text{CLIP}_{\text{hi}}) A)].$$

[**DAPO** / **CLIP_HI**] sets an asymmetric *upper* clip $1 + \text{CLIP_HI}$ (anti-passivity: lets good actions ratchet up faster than bad ones are cut); $\text{CLIP_HI} = 0$ recovers the symmetric PPO clip $1 \pm \text{CLIP}$ (the default).

Value loss and PopArt. The critic regresses the PopArt-normalized return, $L^{\text{vf}} = \text{MSE}(V, \text{popart.normalize}(\dots))$. After each step, `popart.update` rescales `val_out`’s weight/bias so the head’s *raw* predictions are preserved while the running mean/variance track the target distribution ($\beta = \text{POPART_BETA} = 3 \times 10^{-4}$). This keeps value targets $O(1)$ across the huge ± 1000 outcome scale.

Entropy. A per-owned-source mean entropy bonus $\bar{H} = \mathbb{E}[H/n_{\text{owned}}]$ (gate-only by default, document 1) is subtracted with the annealed `ent_coef`.

Total loss.

$$L = \underbrace{p_c}_{\text{warmup}} L^{\text{pol}} + \text{VF_COEF} \cdot L^{\text{vf}} - p_c \cdot \text{ent_coef} \cdot \bar{H} (+ \text{AUX_REWARD_COEF} \cdot L^{\text{aux}}),$$

followed by global-norm grad clipping at `MAX_GRAD_NORM` and a fused-Adam step (skipped if the grad norm is non-finite).

3.1 BC-stability guardrails

PPO is initialized from a sharp BC clone (document 1), so the early updates must not wreck it:

- **Critic-only warmup** (`VALUE_WARMUP_ITERS= 5`): for the first few iters $p_c = 0$, and *after backward* every gradient on a parameter *not* prefixed `val_` is zeroed — so only the critic learns to track the BC policy’s value, and the shared trunk is not dragged by value gradients while the policy is frozen. (Because the aux reward head’s parameters are also non-`val_`, it likewise trains only after warmup.)
- **Linear LR warmup** (`LR_WARMUP_ITERS= 16`): $\text{lr} = \text{LR} \cdot \min(1, (\text{step}+1)/16)$.
- **KL early-stop** (k3 estimator $\widehat{\text{KL}} = \mathbb{E}[(\rho-1)-\log \rho] \geq 0$): a minibatch exceeding `KL_HARD_MULT` $\times$ `KL_TARGET` stops the epoch immediately (`KL_STOP_MINIBATCH`); an epoch whose *mean* KL exceeds `KL_TARGET` ends the update (`KL_EARLYSTOP`).
- **Gate-collapse tripwire** (in `train.py`): if the launch rate’s recent EMA falls below `TRIPWIRE_FRAC` of the early baseline under healthy-looking KL, it warns of a passivity ratchet — the project’s oldest failure mode.

4 The auxiliary reward-prediction head (`AUX_REWARD_PRED`)

This is the “aux head” variant. An optional shallow head off the *shared* trunk regresses the immediate per-step reward r_t as a supervised auxiliary (UNREAL-style), adding $L^{\text{aux}} = \text{MSE}(\hat{r}, r_t)$ weighted by `AUX_REWARD_COEF= 0.25`. The target r_t is exactly the reward signal GAE already consumes (document 1, including the terminal payment). The head is built only when the flag is on, so the network — and every existing checkpoint — is byte-identical when off; when on, the flag

is persisted in the checkpoint `config` blob and the head is rebuilt per-member, so pre-aux league snapshots and invited members still load (document 1’s `LegacyObsAdapter` / `build_from_cfg` plumbing).

4.1 Does the aux head “replace GAE”? No — and here is why a reward head cannot

It is tempting to read the aux head as a GAE substitute. It is not: **GAE runs unchanged**, the critic still produces the baseline, and the aux head is a pure *representation* auxiliary — it adds gradient to the trunk and changes neither the reward, the advantage, nor the critic.

The deeper reason a reward-prediction head cannot stand in for GAE is that **folding a value estimate into the reward is a GAE no-op**. GAE telescopes: with the shaped reward $r'_t = r_t + \gamma \Phi(s_{t+1}) - \Phi(s_t)$ for any state potential Φ , the advantage

$$A_t^\lambda(r') = A_t^\lambda(r) + [\Phi(s_t) - \Phi(s_t)] \text{ (the } \Phi \text{ terms cancel along the } \lambda\text{-sum)}$$

is *unchanged* — this is exactly the policy-invariance of potential-based shaping (the same identity document 1 uses for the shaping reward). So you cannot manufacture a useful baseline by predicting reward and adding it back in: whatever potential-shaped quantity a reward head learns, GAE’s discounted-difference structure cancels it. A learned *baseline* (the critic) is needed precisely because it enters the advantage *outside* that telescoping cancellation ($A_t = \sum (\gamma\lambda)^k \delta_{t+k}$ with δ built from V). The aux head is therefore positioned as what it can actually help: pressuring the shared trunk to encode reward-predictive features, which helps when the RL signal is terminal-dominated — not as a critic replacement.

4.2 Caveat

The aux target r_t *includes* the sparse terminal payment, which overlaps what the critic already predicts at the last step (a large terminal spike on top of the small dense shaping). A *dense-only* variant (regress the shaping channels, exclude the terminal payment) avoids that overlap and is the cleaner representation signal; it is offered but not the current default. See memory `v12-aux-reward-head`.

5 The win-bet head (AUX_WIN_BET) — a stronger aux target

A more decision-relevant alternative to predicting r_t : have the shared trunk *bet* on the eventual game outcome. The same shallow scalar head emits a bet $b_t = \tanh(\hat{r}) \in [-1, 1]$, scored against the episode outcome $z \in [-1, 1]$ (2p ± 1 win/loss; 4p placement) by the linear payoff $b_t \cdot z$; the update minimizes $-\text{AUX_WIN_BET_COEF} \cdot \mathbb{E}[b_t z]$ (bet +1 & win $\rightarrow +1$; bet +1 & lose $\rightarrow -1$; bet -1 & lose $\rightarrow +1$). Like the reward head it is a *pure representation* aux — not in the reward, advantage, or critic — reuses the same shallow head and checkpoint plumbing (mutually exclusive with `AUX_REWARD_PRED`; win-bet takes precedence), and trains only after the value warmup. It pressures the trunk to discriminate winning vs. losing states, a far stronger signal than the immediate reward. The per-step target is the terminal outcome broadcast to every step (`tb["outcome"]`, set in `collect_ppo`); it is consumed by both `ppo_update` and the critic-free `grpo_update`, so it trains under PPO, GRPO, and `ALGO="mc"` (the blockseq MC notebooks enable it).

Caveat: the linear payoff is not a proper scoring rule — its optimum is the hard bet $b = \text{sign}(2p - 1)$, so it elicits the *sign* of the win-probability, not a calibrated p . That is fine for a

representation aux (it forces confident win/loss discrimination); use a log/Brier rule if calibrated probabilities are wanted. See memory `v12-aux-win-bet-head`.

6 Config-knob map

field	default	controls
ALGO	"ppo"	selects this learner (vs "grpo")
GAMMA	0.997	discount
GAE_LAMBDA	0.98	GAE bias/variance trade-off
CLIP	0.2	symmetric PPO clip half-width
CLIP_HI	0.0	[DAPO] asymmetric upper clip $1 + \text{CLIP_HI}$ (0 = symmetric)
RATIO_LENGTHNORM	False	[GSPO] divide the log-ratio by #owned planets
VF_COEF	0.5	value-loss weight
ENT_COEF_START/END	0.005/0.002	entropy anneal (over ENT_DECAY_ITERS= 300)
MINIBATCHES	64	minibatches per epoch
UPDATE_EPOCHS	3	epochs per update
KL_TARGET	0.05	KL early-stop target (k3)
KL_HARD_MULT	1.5	per-minibatch hard-stop multiplier
VALUE_WARMUP_ITERS	5	critic-only (val-grad-masked) warmup iters
LR_WARMUP_ITERS	16	linear LR warmup
USE_POPART	True	output-preserving value-target normalization
AUX_REWARD_PRED	False	build/train the UNREAL reward-prediction head
AUX_REWARD_COEF	0.25	aux reward-MSE weight
AUX_WIN_BET	False	build/train the win-bet head (maximize $b \cdot z$; precedence over reward-pred)
AUX_WIN_BET_COEF	0.25	win-bet aux weight